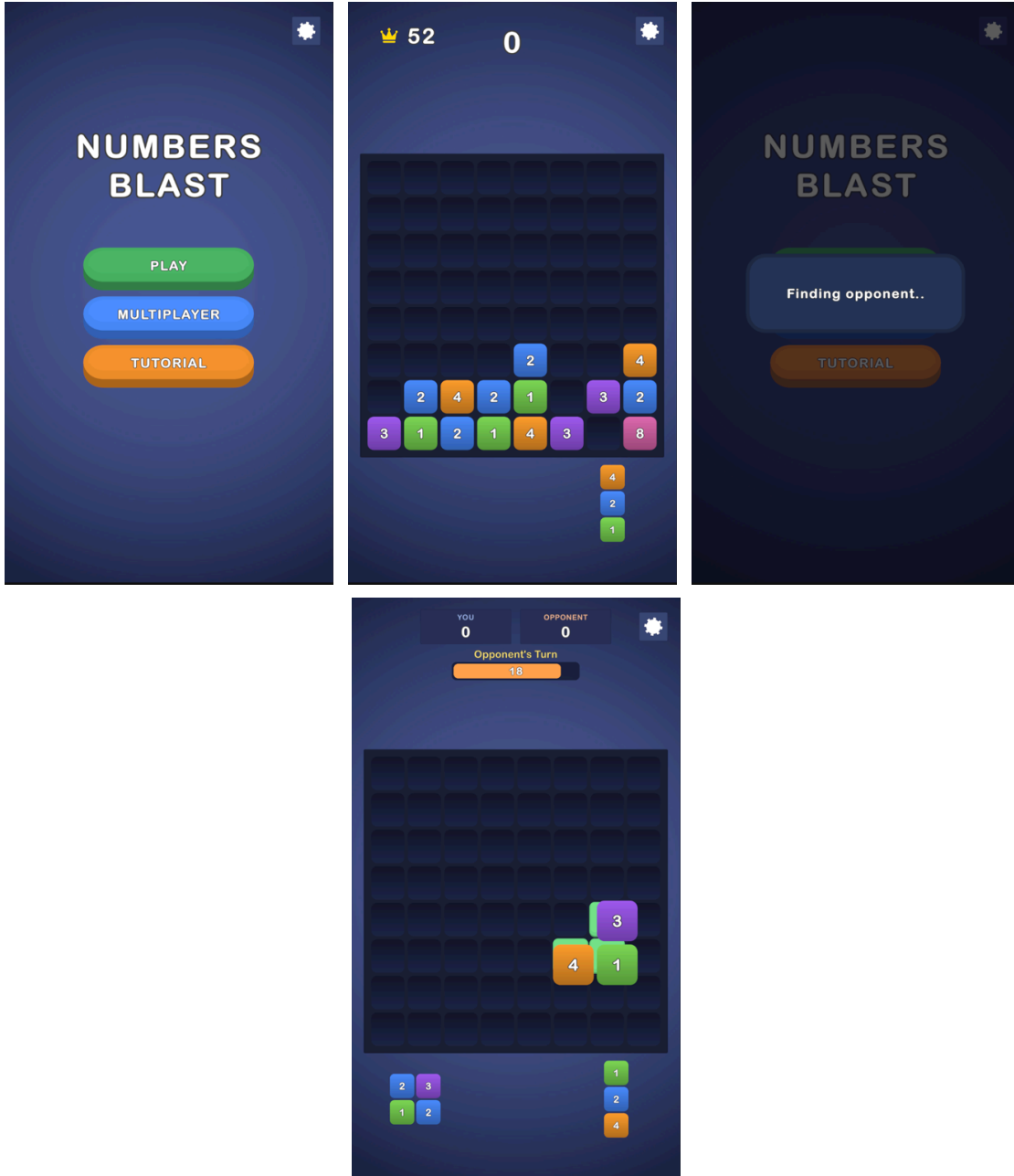


Numbers Blast

Musab Kara

Numbers Blast

Block Blast tarzı, sayı birleřtirmeli bir bulmaca prototipi: yerleřtirilen blok, eř deęerli komřularını kendine katar (zincirleme devam eder), dolan satır/sütunlar temizlenip skor verir. Çekirdek oyunun yanında, inandırıcı bir AI rakibe karřı sahte gerçek-zamanlı bir multiplayer modu ierir.



Nasıl çalıştırılır

1. Depoyu **Unity 6000.3.8f1** ile açın.
2. `Assets/Scenes/Game.unity` sahnesini açıp **Play**'e basın.
3. Testler: *Window* ▶ *General* ▶ *Test Runner* — 25 EditMode + 5 PlayMode.

Mimari

Tek assembly; klasörler namespace'lerle bire bir eşleşir.

Core	Değişmez veri + enum'lar
Data	ScriptableObject'ler (board, parça şekilleri, tutorial adımları)
Gameplay	Saf mantık, UI'sız (board, tray, hamle pipeline'ı, skor)
App	Kompozisyon kökü: <code>GameSessionController</code> + oturum yardımcıları
Presentation	Görseller, animasyonlar, ortak yerleştirme önizlemesi
Input	<code>PieceDragController</code> (fare ve dokunma tek yol)
UI	Menü, skorboard, tur/sayaç, tutorial, oyun sonu, eşleştirme
Tutorial	3 adımlı zorunlu tutorial
Opponent	Maç döngüsü, hamle değerlendirici, insansı sunum
Settings	SFX / müzik / titreşim + ayarlar paneli

Tek doğruluk kaynağı `PlacementService.ApplyMove` (yerleştir → merge → temizle): önizleme, gerçek hamle, AI değerlendirmesi ve fail-state aynı pipeline'ı kullanır — gördüğünüz önizleme ile sonuç yapısal olarak ayrılmaz. `Gameplay` katmanı hiçbir UI/Input tipini referans etmez.

Öne çıkan kararlar

- **Önce merge, sonra clear** — sıra kuralın parçasıdır ve önizleme aynısını gösterir; skor yalnız satır/sütun temizliğinden gelir (kesişim bir kez sayılır).
- **Tray içeriği modeldedir** (`TrayModel`); parça tüketimi tek noktada ve yalnızca hamle doğrulandıktan sonra yapılır.
- **Part 2:** iki turda da görünen 20 sn sayaç, ekranda görünür timeout cezası, tamamen ekran üzerinde oynayan rakip; ayarlar maçı durdurmaz.
- **AI** oyuncuyla aynı pipeline'la değerlendirir; maç içi rubber-band maçları yakın tutar, bariz en iyi hamle asla kaçırılmaz; rakip, oynayacağından daha iyi görünen bir hücrenin üzerinde sahte şekilde oyalanmaz.
- DI framework'ü, service locator, event bus, kullanılmayan interface yok.

Kararların gerekçeleri ve detaylı anlatım: [README.pdf](#)

Bilinen notlar

- Tutorial yalnızca ilk açılışta çalışır; menüden tekrar oynatılabilir.
- Palet dışındaki merge değerleri kararlı bir golden-ratio tonu alır.
- Rakip turu tipik olarak 4–5 saniye sürer ve her zaman 20 saniyelik sayaç dolmadan çok önce biter.

Gelecek geliştirmeler

Rubber-band eşiğine bağlı zorluk seçici · rakip turu için bir üst süre sınırı · floating score yazıları için pooling.

Numbers Blast — Mimari ve Tasarım Kararları

Bu doküman, projedeki temel teknik ve tasarım tercihlerini nedenleriyle birlikte özetler. Amaç kodu tekrar anlatmak değil; alınan kararların arkasındaki yaklaşımı ve özellikle tercih edilmeyen alternatifleri görünür kılmaktır.

1. Genel yapı: tek assembly, klasör = namespace

Core → Data → Gameplay → App → Presentation / Input / UI / Tutorial / Opponent / Settings

Saf oyun mantığı (`Gameplay`) hiçbir UI veya Input tipine referans vermez. Bu sınır ek bir sözleşme katmanıyla değil, namespace ayrımıyla korunur. İlgili katmanda herhangi bir görsel ya da input `using` i bulunmaz; bu durum grep ile de doğrulanabilir. Sistemin tamamını bilen tek sınıf kompozisyon köküdür: `GameSessionController` .

Bu yaklaşımın nedeni şudur: Bu ölçekte asıl risk, oyun mantığının UI katmanına sızarak test edilemez hale gelmesidir. Namespace sınırı, ek karmaşıklık yaratmadan bu riski azaltır. Saf oyun katmanı sayesinde kuralların tamamı sahneye ihtiyaç duymadan, çok hızlı EditMode testleriyle güvence altına alınabilir.

DI framework, service locator veya event bus tercih edilmedi. Bu ölçekte bu yapıların sağlayacağı fayda, getirecekleri ek karmaşıklıkla karşılamazdı. Service locator ve global event bus, akışı görünmez hale getirebilir. Bunun yerine explicit referanslar tercih edildi; bağımlılıklar `Initialize` üzerinden açıkça verildi. Böylece “kim kimi çağırıyor?” sorusunun cevabı doğrudan kod üzerinden takip edilebilir. Projede kullanılmayan interface veya soyutlama bulunmaması bilinçli bir karardır; “no unnecessary classes” beklentisi tasarım sürecinin temel hedeflerinden biri olarak ele alınmıştır.

2. Tek hamle pipeline’ı — projenin omurgası

`PlacementService.ApplyMove(board, piece, anchor)` tek bir sorumluluğa sahiptir:

yerleştir → merge'leri çöz → dolu satırları temizle.

Bu metot dört farklı yerde kullanılır: canlı sürükleme önizlemesi, gerçek hamle, AI aday değerlendirmesi ve fail-state kontrolü.

Bu kararın temel nedeni kural tutarlılığıdır. Kural tek yerde yaşadığında önizleme, AI değerlendirmesi ve gerçek sonuç birbirinden ayrılamaz. Oyuncuya gösterilen şey ile hamle sonrası oluşan sonuç yapısal olarak aynı olur. Bu güvence yalnızca testlere değil, doğrudan mimariye dayanır. Alternatif yaklaşımda her tüketici kendi kural mantığını taşırdı; bu da üç ayrı kopya kural, daha fazla bakım yükü ve zamanla kaçınılmaz davranış farkları anlamına gelirdi.

Önizleme scratch board üzerinde çalışır. Çünkü `BoardModel` düz bir `int[]` yapısına sahiptir ve `CopyFrom` ile kopyalanması ucuzdur. Gerçek simülasyonu geçici bir board üzerinde çalıştırmak, ikinci bir “tahmini highlight” mantığı yazmaktan hem daha doğru hem de daha güvenlidir. Bunun testi de vardır: önizleme gerçek board'u asla değiştirmez.

3. Kural kararları

Önce merge, sonra clear çalışır. Bu sıra deterministiktir ve öngörülebilir bir davranış üretir. Bir merge işlemi, dolu görünen bir satırı boşaltabilir. Bu bir yan etki değil, kuralın bilinçli tanımıdır. Önizleme de aynı sırayı kullandığı için oyuncu beklenmeyen bir sonuçla karşılaşmaz. Ters sıra denendiğinde, özellikle zincirleme merge ihtimallerinde davranış daha belirsiz hale geliyordu.

Merge skor vermez. Skor yalnızca satır veya sütun temizliğinden gelir. Satır ve sütun kesişimindeki hücre ise yalnızca bir kez sayılır. Böylece puanlama tek, anlaşılır ve açıklanabilir bir kaynaktan beslenir.

Parça üretimi sırasında, parça içindeki komşu hücrelerde eş değerlerin oluşmasına izin verilmez. Böylece parça daha doğduğu anda kendi kendine merge olamaz. Oyuncunun gördüğü her merge, kendi hamlesinin sonucu olarak ortaya çıkar.

4. Durum yönetimi

Oyun akışı küçük ve net bir state alanıyla yönetilir:

Menu / PlayerTurn / Resolving / OpponentTurn / GameOver

State ataması tek bir noktadan geçer. Bu sayede oyun akışındaki geçişler dağılmaz ve takip edilebilir kalır.

Ayarlar paneli, oyun state'ini bozarak değil, ayrı bir `InputGate` üzerinden input'u keserek çalışır. Panel herhangi bir anda açılrsa bile gerçek oyun durumu ezilmez veya yanlış bir state'e düşmez.

Tray'in içeriği görsel katmanda değil, oturuma ait `TrayModel` içinde tutulur. Fail-state tespiti ve AI değerlendirmesi bu modeli okur. Parça tüketimi tek noktada ve yalnızca hamle doğrulandıktan sonra yapılır. Model ve görsel birlikte güncellendiği için “yarım tüketim” gibi hata sınıfları yapısal olarak engellenir.

Kompozisyon kökü büyüdüğünde üç odaklı düz sınıfa ayrıldı: HUD düzeni, oyun sonu koreografisi ve input kilidi. Bu sınıfların MonoBehaviour olmaması özellikle tercih edildi. Böylece sahnedeki referans bağlantıları hiç değişmeden kaldı; bölme işlemi davranışı veya sahneyi riske atmadan yapılabilirdi.

5. Part 2: multiplayer hissi tutarlılıktan doğar

İkinci bölümde amaç gerçek bir online multiplayer sistemi kurmak değil, tutarlı ve inandırıcı bir multiplayer hissi oluşturmaktır. Bunun için tek bir büyük numara yerine, birbiriyle uyumlu küçük detayların toplamı tercih edildi: paylaşılan board ve tray, sahte “Finding opponent...” ekranı, iki turda da görünen 20 saniyelik geri sayım, rakip turunda farklı renk kullanımı, ekranda görünür “Time’s up! –5%” cezası ve hamlesini doğrudan ekran üzerinde yapan bir rakip.

Ayarlar paneli maçı durdurmaz. Çünkü gerçek bir online rakip oyuncuyu beklemezdi. Bu nedenle panel yalnızca yerel oyuncunun girişini kilitler; maç akışı kendi içinde devam eder.

AI tarafında hedef kusursuz oynamak değil, iyi ve inandırıcı oynamaktır. Her aday hamle, oyuncuyla aynı pipeline üzerinden değerlendirilir. AI için ayrı bir kural sistemi yoktur. En iyi adaylar arasından ağırlıklı-rastgele seçim yapılır. Bunun üzerine üç inandırıcılık katmanı eklenmiştir:

1. Maç içi rubber-band: Seçim genişliği anlık skor farkına göre değişir. Öndeyken daha gevşek oynar, gerideyken daha güçlü hamlelere yönelir. Profil veya kalıcı zorluk sistemi gerektirmez; tek maç içinde bile hissedilir.
2. Bariz hamle kuralı: En iyi aday açık farkla öndeyse, örneğin net bir satır temizliği sağlıyorsa, AI bu hamleyi kaçırmaz. Böylece “temizliği görmesine rağmen anlamsız yere oynayan robot” hissi engellenir.
3. Dürüstlük kuralları: Rakip, gerçekte oynayacağından daha iyi görünen bir hücrenin üzerinde sahte şekilde gezinmez. Tereddüt süresi ve deneme sayısı tamamen rastgele değildir; kararın gerçekten ne kadar yakın olduğuna göre belirlenir. Bariz hamlede doğrudan gider, yakın kararda düşünür. Böylece kararlılık da tereddüt kadar insansı görünür.

Sunum, karardan ayırdır. Saf ve seed’lenebilir bir plancı, hamleyi “beat” listesine çevirir: düşünme, deneme, nadiren yanlış bırakma, yanlış taşı alıp vazgeçme gibi adımlar bu listede tanımlanır. Görsel katman yalnızca bu planı oynatır. Bu ayrım sayesinde koreografi mantığı da unit-test edilebilir hale gelir.

6. Test stratejisi

Test yapısı iki katmandan oluşur ve her katmanın amacı farklıdır.

25 EditMode testi kural katmanını güvence altına alır. Merge zinciri, skor, kesişim hesabı, önizlemenin gerçek board’a dokunmaması, ceza matematiği, fail-state sınırları, spawn kuralı ve plancı davranışları bu testlerle doğrulanır. Bu testler sahneye ihtiyaç duymaz ve CI için uygundur.

5 PlayMode testi ise gerçek sahneyi açarak oyunu uçtan uca çalıştırır. Solo akış, Vs AI başlangıcı, timeout sonrası turun devredilmesi ve ayar kilidinin temizlenmesi bu testlerle kontrol edilir.

Geliştirme sırasında bulunan her gerçek hata bir regression testine dönüştürüldü. Bu nedenle test kapsamı yalnızca teorik bir güvence değil, aynı zamanda projenin hata geçmişinin de haritasıdır.

7. Performans — Android hedefi

Android hedeflendiği için, performans açısından kritik kod yollarında (hot path) LINQ kullanılmadı. Önizleme yalnızca hedef hücre değiştiğinde yeniden hesaplanır; parça aynı hücrenin üzerinde tutulduğu sürece GC allocation oluşmaz. Sayaç yazısı yalnızca saniye değeri değiştiğinde güncellenir; her karede yeni string üretilmez. Satır temizliğinde oynayan parlama efektleri her seferinde yeniden yaratılmaz; object pooling ile yeniden kullanılır. AI, her aday için yeni board ayırmak yerine tek bir scratch board'u yeniden kullanır. `Update` yalnızca tur sayacı ve safe-area gibi gerekli alanlarda çalışır.

Canvas, Expand modunda ölçeklenir: 1080×1920'lik tasarım alanı her ekran oranında bütünüyle görünür kalır; uzun ekranlarda fazlalık dikeye, geniş ekranlarda yanlara boşluk olarak dağılır.

8. Bilinçli ertelenenler

Zorluk seçici, rakip turu için bir üst süre sınırı ve floating score yazıları için de pooling uygulanması bilinçli olarak ertelendi. Rubber-band eşiği zaten Inspector üzerinden ayarlanabilir durumdadır. Bu başlıkların hiçbiri unutulmadı; yalnızca bu ölçek için karşılığı sınırlı olan ek karmaşıklıklar olarak değerlendirildi ve README'de açıkça listelendi.